TECHNICAL WHITEPAPER

# Verifiable Event Infrastructure for *Autonomous Commerce*

A cryptographically anchored sequencing layer that gives AI agents a shared, tamper-evident source of truth — enabling trustless ordering, Merkle-provable audit trails, and on-chain settlement without requiring trust in any single operator.

| PROTOCOL | RUNTIME | SETTLEMENT | LICENSE |
| --- | --- | --- | --- |
| VES v1.0 | Rust / Tokio | Set Chain L2 | MIT |

EXECUTIVE SUMMARY

The StateSet Sequencer is a **production-grade event ordering service** that implements the Verifiable Event Sync (VES) v1.0 protocol. It assigns gap-free, cryptographically signed sequence numbers to events produced by autonomous commerce agents—then anchors batch Merkle commitments to an Ethereum L2 for **independently verifiable finality**. The result is an infrastructure primitive that makes multi-agent commerce auditable, non-repudiable, and settlement-ready without trusting any single party.

CONTENTS

The Ordering Problem in Agent Commerce 01

System Architecture 02

The VES v1.0 Protocol 03

Ordering Semantics & Finality 04

Merkle Commitments & Inclusion Proofs 05

On-Chain Anchoring & Anti-Equivocation 06

End-to-End Encryption 07

Zero-Knowledge Compliance 08

The x402 Payment Protocol 09

Performance & Resilience 10

Security Model 11

01

# The Ordering Problem in Agent Commerce

*Why wall clocks fail and databases lie*

Modern commerce is rapidly becoming multi-agent and offline-first. Autonomous AI agents manage inventory, process orders, handle returns, and negotiate pricing — often concurrently and without human oversight. When Agent A modifies inventory at the same moment Agent B processes a purchase against that same stock, a fundamental question arises: *whose operation takes precedence?*

Wall-clock timestamps are unreliable across distributed systems. Clock skew, NTP drift, and timezone ambiguity make temporal ordering non-deterministic. Traditional databases provide ordering within a single node but are opaque to external auditors — an administrator can silently reorder, modify, or drop events without detection.

In a world where commercial disputes involve autonomous agents operating across organizational boundaries, the infrastructure itself must provide verifiable guarantees. Three capabilities are non-negotiable:

THE THREE REQUIREMENTS

**Deterministic ordering** — a single canonical sequence that all participants agree on, independent of wall clocks.

**Non-repudiation** — cryptographic proof that an event was accepted, which the sequencer cannot later deny.

**Independent auditability** — any third party can verify the complete history without trusting the operator.

The StateSet Sequencer was built to satisfy all three. It is the canonical clock for distributed autonomous commerce — a cryptographic notary that assigns tamper-evident sequence numbers, issues signed receipts, and periodically anchors batch Merkle commitments on-chain for independently verifiable finality.

02

# System Architecture

*Layered design for production durability*

The sequencer is implemented in Rust on the Tokio async runtime, chosen for memory safety without garbage collection pauses and predictable tail latencies. The architecture follows a strict layered separation: domain types, infrastructure traits with pluggable backends, API surface, and cross-cutting concerns.

```
┌──────────────────────────────────────────────────────────────┐
│                          API Surface                         │
│   HTTP/REST (Axum)                    gRPC v1/v2 (Tonic)     │
│   POST /api/v1/ves/events/ingest      IngestVesEvents()      │
│   GET  /api/v1/ves/commitments        StreamEvents()         │
│   POST /api/v1/ves/proofs             SubscribeEntity()      │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────┴───────────────────────────────────┐
│                       Service Layer                          │
│  VesSequencer  │  CommitmentEngine  │  AgentKeyRegistry      │
│  Projector     │  SchemaValidator   │  PayloadEncryption     │
│  RateLimiter   │  CircuitBreaker    │  AuditLogger           │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────┴───────────────────────────────────┐
│               PostgreSQL  (Source of Truth)                  │
│  ves_events  │  sequence_counters  │  batch_commitments      │
│  agent_keys  │  api_keys  │  schemas  │  vault  │  audit_log │
└──────────────────────────┬───────────────────────────────────┘
                           │
          ┌────────────────┴────────────────┐
          ▼                ▼                ▼
   STARK Prover       Set Chain L2       Anchor Worker
   (ZK Proofs)      (Commitments)     (Gas Optimization)
```

DOMAIN

### Pure Types

Event envelopes, commitments, proofs, and payment intents defined as framework-agnostic Rust types with serde serialization.

---

INFRASTRUCTURE

### Trait-Driven Storage

PostgreSQL for production, SQLite for testing. Connection pooling, read/write splitting, and session-level timeout guarantees.

---

CRYPTO

### Hermetic Signing

Domain-separated SHA-256 hashing, Ed25519 signatures, AES-256-GCM encryption, and HPKE key wrapping — all spec'd to the byte.

---

API

### Dual Interface

REST via Axum for simplicity, gRPC via Tonic for streaming. Both enforce identical validation, rate limiting, and timeout semantics.

---

AUTH

### Layered Security

API key and JWT authentication, per-tenant rate limiting, pool health-aware load shedding, and security headers on every response.

---

ANCHOR

### L2 Settlement

Two-phase anchoring to Set Chain via Alloy, with circuit breaker protection and exponential backoff for blockchain reliability.

> *Auditors don't need to trust the sequencer operator, the database, or any single party. They only need to verify the math—and the math is public.*

DESIGN PRINCIPLE — VES V1.0 SPECIFICATION

03

# The VES v1.0 Protocol

*Verifiable Event Sync — a cryptographic contract between agents and the sequencer*

At the heart of the StateSet Sequencer is the Verifiable Event Sync protocol, a specification that defines exactly how events are hashed, signed, sequenced, receipted, and committed. Every byte of every hash is deterministic and reproducible across any language implementation.

## The Event Envelope

Every event entering the system is wrapped in a `VesEventEnvelope` — a structure containing both agent-provided fields (identity, payload, signature) and sequencer-assigned fields (sequence number, acceptance timestamp). The agent signs a deterministic binary encoding of its fields before submission; the sequencer verifies this signature before assigning ordering.

| FIELD | SOURCE | PURPOSE |
|---|---|---|
| **`event_id`** | Agent | Globally unique identifier (UUID v4) |
| **`tenant_id / store_id`** | Agent | Stream identity & isolation boundary |
| **`entity_type / entity_id`** | Agent | Domain aggregate (order, inventory, product) |
| **`payload_plain_hash`** | Agent | SHA-256 of JCS-canonicalized payload with domain prefix |
| **`agent_signature`** | Agent | Ed25519 over deterministic binary encoding |
| **`sequence_number`** | Sequencer | Canonical ordering position (gap-free, monotonic) |
| **`sequenced_at`** | Sequencer | Acceptance timestamp (observational, not authoritative) |

## Domain-Separated Hashing

All cryptographic operations use domain-prefixed hashing to prevent cross-protocol collision. The payload hash, for instance, prepends `VES_PAYLOAD_PLAIN_V1` before hashing the RFC 8785 JSON Canonicalization Scheme output. Ten distinct domain prefixes cover payloads, signatures, Merkle leaves, Merkle nodes, receipts, stream identifiers, encryption AAD, and more.

> **WHY JCS (RFC 8785)?**
>
> JSON Canonicalization eliminates the "same data, different bytes" problem. Two implementations in different languages producing semantically identical JSON will compute bit-identical hashes. This is essential for cross-platform signature verification.

## Sequencer Receipts

Upon accepting an event, the sequencer returns a signed receipt binding the event identity to its assigned sequence number. This receipt is non-repudiable: an agent can prove acceptance even if the sequencer later attempts censorship. The receipt preimage includes `VES_RECEIPT_V1`, the tenant and store identifiers, the event ID, sequence number, and the original event signing hash.

04

# Ordering Semantics & Finality

*One number to rule them all*

The `sequence_number` is the single canonical ordering primitive. Agent-provided `created_at` timestamps are signed but never used for ordering. The sequencer's own `sequenced_at` is observational metadata useful for monitoring but not authoritative for replay or state reconstruction.

For each stream (identified by the `tenant_id` and `store_id` pair), sequence numbers are strictly increasing by one with no gaps. This invariant is enforced atomically via PostgreSQL `SELECT ... FOR UPDATE` on per-stream counters, guaranteeing linearizable ordering even under concurrent ingestion from multiple agents.

---

*The only authority on what happened and in what order is the sequence number. Everything else—timestamps, wall clocks, network arrival order—is advisory.*

ORDERING INVARIANT

---

Events are idempotent by `event_id`: resubmitting an identical event returns the same sequence number, while resubmitting the same ID with different content is rejected. An optional `command_id` field provides intent-level deduplication for higher-level workflows.

## The Two-Phase Finality Model

SOFT FINALITY

**< 100ms**

Signed receipt returned to agent. Non-repudiable proof of acceptance.

→

HARD FINALITY

**1–5 min**

Batch Merkle root anchored on L2. Independently verifiable by any auditor.

| USE CASE | REQUIRED FINALITY | RATIONALE |
|---|---|---|
| **Order confirmation display** | Soft | Receipt provides instant, signed acknowledgement |

| USE CASE | REQUIRED FINALITY | RATIONALE |
| --- | --- | --- |
| **Inventory reservation** | Soft | Sequence number establishes priority; optimistic concurrency handles conflicts |
| **Trade finance settlement** | Hard | On-chain anchor provides tamper-evident proof for multi-party agreements |
| **Regulatory audit trail** | Hard | Merkle proofs against on-chain roots enable independent verification by auditors |

05

# Merkle Commitments & Inclusion Proofs

*Compact, verifiable batch integrity*

The sequencer periodically constructs Merkle trees over contiguous ranges of events, producing a single 32-byte root hash that commits to the entire batch. This root is the anchor point for both on-chain settlement and independent audit.

## Batch Commitment Structure

Each `VesBatchCommitment` covers a contiguous sequence range and includes the Merkle root, sequence boundaries, event count, and a reference to the previous batch's root. This chaining creates an append-only commitment log that prevents retroactive modification.

## Leaf Construction

Each leaf in the Merkle tree is computed from a domain-separated hash over the event's stream identity, canonical sequence position, event signing hash, and actual signature bytes. This binds every leaf to its position, content, and authorship — preventing reordering, content modification, and signature substitution.

1

**LEAF HASH**

SHA-256 over domain prefix, stream ID, sequence number, signing hash, and signature bytes

2

**TREE BUILD**

Pad to power-of-two with domain-separated padding leaves. Internal nodes use `VES_NODE_V1` prefix.

3

**ROOT**

Single 32-byte hash at tree apex. Commits to every event in the batch.

4

**PROOF**

O(log n) sibling hashes from leaf to root. Any verifier recomputes and compares.

**VERIFIER INDEPENDENCE**

Any third party can verify event inclusion: fetch the event data, recompute the leaf hash, walk the proof path with provided siblings, and confirm the result matches the on-chain Merkle root. No trust in the sequencer required.

06

# On-Chain Anchoring & Anti-Equivocation

*Blockchain-verifiable settlement on Set Chain L2*

The final trust anchor is the blockchain. Batch Merkle roots are submitted to the `StateSetAnchor` smart contract on Set Chain L2 (EVM-compatible), creating an immutable, publicly queryable record that any auditor can verify without the sequencer's cooperation.

## The Fork Problem & Commitment Chaining

A dishonest sequencer could potentially anchor two conflicting Merkle roots for the same stream — presenting one history to auditors and another to merchants. VES prevents this through **commitment chaining**: each new batch includes the previous batch's root hash, and the smart contract enforces continuity.

01 **Chain rule** — `prev_events_root` must match the current on-chain head for that stream

02 **Continuity rule** — `sequence_start` must equal the previous batch's `sequence_end + 1`

03 **Non-empty rule** — `sequence_end` must be greater than or equal to `sequence_start`

04 **Genesis rule** — First commitment uses `prev_events_root = 0x000...000` and `sequence_start = 0`

Any fork attempt is rejected by the contract and publicly detectable. The result is a single, append-only commitment chain per stream — the on-chain equivalent of a tamper-evident seal.

## Resilient Submission

On-chain submission is protected by a circuit breaker (opens after 5 failures, 60-second cooldown) and exponential backoff retry (up to 10 attempts, 1 second to 5 minute delay range with 50% jitter). The anchor service supports multiple L2 targets: Set Chain, Arbitrum, Optimism, and Base.

07

# End-to-End Encryption

*Sequencer-blind ordering for sensitive payloads*

For privacy-sensitive events — payment details, customer PII, trade terms — VES supports the VES-ENC-1 encryption scheme. Agents encrypt payloads locally with AES-256-GCM before submission. The sequencer orders and commits encrypted events without ever accessing plaintext.

A per-event random Data Encryption Key (DEK) is generated, the payload is encrypted with a 16-byte random salt prepended, and the DEK is wrapped for each authorized recipient using HPKE (Hybrid Public Key Encryption) with X25519 key agreement. The sequencer sees only ciphertext.

SECURITY PROPERTIES

**Sequencer-blind:** Cannot access plaintext.
**Context-bound:** AAD prevents ciphertext swapping between events.
**Salt-protected:** Prevents dictionary attacks on low-entropy payloads.
**Forward-secret:** Per-event random DEKs; compromise of one key does not expose others.
**Dual-hash binding:** Both plain and cipher hashes are signed, preventing substitution.

Critically, both `payload_plain_hash` and `payload_cipher_hash` are bound to the agent's signature. For plaintext events, the cipher hash is 32 zero bytes. This prevents an attacker from replacing an encrypted payload with a different ciphertext while keeping a valid signature.

08

# Zero-Knowledge Compliance

*Proving correctness without revealing data*

Regulators need proof of compliance — correct VAT calculation, AML screening, export control adherence — but revealing the underlying transaction data creates privacy and competitive risks. STARK proofs resolve this paradox.

BATCH LEVEL

### Validity Proofs

STARK proofs over entire batches demonstrating that all events in a commitment satisfy specified policy constraints. Submitted via `/api/v1/ves/validity-proofs` and anchorable on-chain alongside batch commitments.

PER EVENT

### Compliance Proofs

Individual event proofs for targeted regulatory queries. An auditor can verify that a specific transaction complied with rules without the sequencer or merchant revealing the transaction amount, counterparty, or terms.

WHY STARKS OVER SNARKS?

STARKs require no trusted setup ceremony, provide 128-bit conjectured security, offer post-quantum resistance through hash-based cryptography, and verify in $O(\log^2 N)$ time. Proof sizes of 100–200 KB are acceptable for batch-level compliance attestation.

09

# The x402 Payment Protocol

*Agent-to-agent settlement with gas-optimized batching*

When autonomous agents transact, payment settlement must be as programmable as the commerce itself. The x402 protocol integrates payment intent sequencing directly into the event stream, enabling batched, gas-efficient settlement on-chain.

1

**INTENT**

Agent creates Ed25519-signed `X402PaymentIntent` specifying amount, asset, network, and recipient

2

**SEQUENCE**

Sequencer validates signature and assigns canonical sequence number to the payment event

3

**BATCH**

Background worker assembles 100–1,000 payment intents into a single L2 transaction

4

**SETTLE**

`SetPaymentBatch` contract executes aggregated ERC-20 transfers on Set Chain

| | |
|---|---|
| **6**<br>SUPPORTED NETWORKS | **6**<br>SUPPORTED ASSETS |
| **1,000**<br>MAX BATCH SIZE | **~5x**<br>GAS SAVINGS PER PAYMENT |

The protocol supports USDC, USDT, DAI, ETH, StateSet USD (ssUSD), and Wrapped ssUSD (ERC-4626 yield-bearing) across Set Chain, Base, Arbitrum, Optimism, Arc, and Ethereum mainnet. Payment intents carry the same cryptographic guarantees as commerce events: signed, sequenced, and anchorable.

# Performance & Resilience

*Engineering for production durability*

| | |
|---|---|
| **5K** evt/s<br>PEAK THROUGHPUT | **8** ms<br>P50 LATENCY |
| **45** ms<br>P95 LATENCY | **95** ms<br>P99 LATENCY |

The system achieves 2,500–5,000 events per second with sub-10ms median latency and sub-100ms P99 through careful engineering of the hot path: connection pooling with separate read/write pools, LRU caching with stale-while-revalidate refresh, and session-level database timeouts (30s statement, 60s idle-in-transaction, 10s lock).

**Caching architecture:** Six specialized LRU caches (commitments, proofs, VES commitments, VES proofs, agent keys, and schemas) serve read-heavy workloads from memory. The read path uses a fast RwLock read-lock for fresh hits, escalating to a write-lock only for stale entries or evictions. A stampede delay prevents thundering-herd refresh patterns.

**Circuit breaker:** External service calls (particularly L2 anchoring) are protected by a circuit breaker that opens after 5 failures, transitions to half-open after a 60-second cooldown, and requires 2 consecutive successes to close. Exponential backoff with jitter prevents retry storms.

**Graceful shutdown:** A `ShutdownCoordinator` distributes signals to all background tasks (pool monitor, metrics collector, anchor worker, DLQ cleanup). Both HTTP and gRPC servers use graceful shutdown hooks to drain in-flight requests before exiting.

**Observability:** Prometheus metrics at `/metrics`, structured JSON logging, OpenTelemetry distributed tracing, and `#[instrument]` spans on every API handler provide full operational visibility.

EXHIBIT A — RESILIENCE STRATEGY BY FAILURE DOMAIN

| FAILURE DOMAIN | STRATEGY | CONFIGURATION |
|---|---|---|
| **Database (transient)** | Exponential backoff retry | `5 retries, 50ms-5s, decorrelated jitter` |
| **Database (sustained)** | Pool health load-shedding | 503 with Retry-After when pool health is critical |
| **L2 anchor (transient)** | Retry with exponential backoff | `10 retries, 1s-5min, 50% jitter` |
| **L2 anchor (sustained)** | Circuit breaker + DLQ | Open after 5 failures, dead-letter with background cleanup |
| **Client overload** | Per-tenant rate limiting | 429 with Retry-After header (configurable per-minute limit) |
| **Request timeout** | Tower middleware | `HTTP: 30s, gRPC: 30s (configurable)` |

> *Every byte of every hash is deterministic and reproducible across any language implementation. Cross-platform test vectors ensure identical results in JavaScript, Rust, Python, Go, and any future client.*

PROTOCOL INTEROPERABILITY GUARANTEE

11

# Security Model

*Defense in depth from the application boundary to the database session*

AUTHENTICATION

### Layered Auth

API key (SHA-256 hashed at rest) and JWT (HMAC-SHA256 with issuer/audience validation) authentication. Rate limiting per tenant with bounded LRU memory. Pool-aware load shedding returns 503 before database pressure cascades.

VALIDATION

### Input Boundaries

All string fields validated at ingestion: entity_type (128 chars), entity_id (512), event_type (256). Request bodies capped at 10MB, batches at 1,000 events, individual payloads at 1MB. SQL queries carry safety-cap LIMIT clauses throughout.

ERROR HANDLING

### Information Discipline

Zero production `unwrap()` calls. All 500 responses use generic messages via `internal_error()` and `grpc_internal_error()` helpers that log real errors server-side. No stack traces, query details, or internal state leak to clients.

HEADERS

### Response Hardening

Security headers on every response: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Cache-Control: no-store`. CORS exposes `x-request-id`, `x-error-code`, and `Retry-After` for SPA clients.

AUDIT TRAIL

Schema mutations (register, update, delete), agent key operations, and administrative actions are audit-logged via `AuditLogBuilder` with actor identity, timestamp, and operation details. Write failures are logged at WARN level — never silently swallowed.

# Roadmap

*From centralized sequencing to permissionless verification*

The current architecture positions the sequencer as a trusted-but-verifiable operator. The roadmap progressively removes trust assumptions while expanding capability.

EXHIBIT B — DEVELOPMENT TRAJECTORY

| PHASE | HORIZON | CAPABILITIES |
|---|---|---|
| **Near-Term** | Q1–Q2 2026 | Multi-sequencer consensus for fault tolerance. HSM integration for hardware-backed key storage. Real-time WebSocket/SSE streaming push. |
| **Mid-Term** | Q3–Q4 2026 | Decentralized on-chain key registry. Cross-stream platform-level Merkle commitments. Automated STARK compliance proof generation. |
| **Long-Term** | 2027+ | Permissionless sequencing with economic incentives (staking, slashing). Cross-chain anchoring to multiple L2s simultaneously. Formal verification of protocol properties. |

*The endgame is infrastructure that requires no trust at all—where sequencing is permissionless, verification is automatic, and the protocol's correctness is machine-checked.*

LONG-TERM VISION

CONCLUSION

The StateSet Sequencer is not simply a database with an API. It is a **cryptographic notary for autonomous commerce**—an

infrastructure primitive that makes multi-agent transactions auditable, non-repudiable, and settlement-ready. By combining deterministic event ordering, Ed25519 signed receipts, Merkle-committed batch proofs, STARK compliance attestation, and on-chain anchoring, it creates a system where **trust is replaced by verification**. And verification is available to everyone.

**STATESET**

Verifiable infrastructure for autonomous commerce.
VES v1.0 · Sequencer v0.2.6 · MIT License

This document describes the StateSet Sequencer architecture and VES v1.0 protocol as of March 2026. Specifications and implementation details are subject to change.